Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 16:
Elezione del Leader IPC (FloodMax e LCR)

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement

L'obiettivo di quest'esperienza risiede nella necessità di gestire dinamicamente network anonimi (o semi-anonimi) allo scopo di individuare tra N Nodi di eguale importanza colui che andrà ad assumere le vesti di **Leader del Grafo**. Come mezzo di propagazione dei pacchetti manteniamo l'ambiente POSIX asincrono testato per l'Hands-On 15: un File System su cui vengono mappati pseudo-device esposti tramite la Syscall `mkfifo`. Questa architettura consente di avere N Processi totalmente isolati in RAM (nessuna locazione globale in comune) che concorrono all'elezione passandosi Stringhe lungo l'anello delle pipe.

2 Stato dell'Arte

Abbiamo implementato ben due varianti protocollari, in base alle informazioni di topologia:

2.1 Algoritmo FloodMax (Grafo generico)

Il **FloodMax** è una variante dell'algoritmo Flood studiata per grafi ciclici non ordinati, in cui la dimensione diametrale della Rete (ossia il cammino minimo più lungo tra ogni coppia) sia un fattore noto. Si fonda su una *Sincronicità per Round*:

- L'agente inizializza il 'max_uid' salvando in un buffer il suo UID Univoco.
- Ad ogni giro di orologio, l'agente emette il 'max_uid' a tutti i suoi vicini uscenti ed entra in uno stato bloccante attendendo la lettura da tutti i suoi archi entranti.
- Se dalle pipe riceve un valore più alto del suo, va a sovrascrivere il 'max_uid'.
- Allo scadere dell'intero parametro diametrale tutti gli agenti nel sistema avranno lo stesso esatto valore nel 'max_uid'. Se quello equivaleva originariamente al proprio UID Univoco nativo si incorona "Leader".

2.2 Algoritmo LeLann-Chang-Roberts (Grafo Anello Unidirezionale)

Acronimo del triumvirato *LCR*, esso è il protocollo cardine nelle reti disposte a corona in maniera unidirezionale. Non necessita di round bloccanti né si cura del diametro del grafo. La state transition function è rigorosa all'esame dei pacchetti asincroni "volanti":

- **Se UID Entrante > Mio**: Inoltra UID Entrante ed esci dalla competizione.
- **Se UID Entrante < Mio**: Scarta il messaggio (purga l'infezione, impedendo loop lenti).
- **Se UID Entrante == Mio**: Sono IL LEADER eletto senza ombra di dubbio!

3 Metodologia ed Implementazione

Nel codice nativo in linguaggio C, `lcr.c` genera 5 FIFO Unidirezionali ($0 \to 1, 1 \to 2, \dots$) avvalendosi di `fork` per istanziare i componenti. Inoltre inserisce dinamicamente una riga di codice in sospensiva `sleep(1)` dopo la conversione `sscanf` interna al ciclo d'ascolto per facilitare l'occhio umano alla visualizzazione dell'elezione real-time.

Stralcio Regole LCR

```c
if (recv_uid > my_uid) {
    write(fd_out, buf, strlen(buf)+1); // Inoltro e divento passivo.
    is_participant = 0;
} else if (recv_uid < my_uid) {
    // Scarto
} else {
    // Il mio token non e' stato decapitato ed e' tornato a me!
    sprintf(msg, "LEADER:%d", my_uid);
    write(fd_out, msg, strlen(msg)+1);
}
```

3.1 Istruzioni di Esecuzione

L'applicativo viene gestito interamente tramite Makefile:

Build Run

```
$ make
$ ./programma
```

4 Risultati

Invocando la compilazione nativa globale 'make all', le prove emettono outputs teoricamente accurati.

4.1 Output FloodMax *(UID Vincente Test = 50)*

Algoritmo di Elezione LCR - Risultati

```
--- FloodMax Algorithm (Generic Graph diam=2) ---

[NODO 0] ROUND 1: Mio Max= 10. Invio ai vicini...
[NODO 1] ROUND 1: Mio Max= 50. Invio ai vicini...
[NODO 2] ROUND 1: Mio Max= 20. Invio ai vicini...
[NODO 3] ROUND 1: Mio Max= 30. Invio ai vicini...
[NODO 3] ROUND 2: Mio Max= 50. Invio ai vicini...
[NODO 2] ROUND 2: Mio Max= 30. Invio ai vicini...
[NODO 0] ROUND 2: Mio Max= 50. Invio ai vicini...
[NODO 1] ROUND 2: Mio Max= 50. Invio ai vicini...
[NODO 2] E' emerso un Leader globale con UID 50. Io mi
   arrendo.
[NODO 0] E' emerso un Leader globale con UID 50. Io mi
   arrendo.
[NODO 3] E' emerso un Leader globale con UID 50. Io mi
   arrendo.
>>> [NODO 1] SONO IL LEADER! (UID 50) <<<
```

Risultato garantito nei tempi stabiliti: due round netti per eleggere correttamente l'agente detentore della chiave d'ordine massimale 'UID 50'.

4.2 Output LCR *(UID Vincente Test = 23)*

Algoritmo di Elezione Bully - Risultati

```
--- LCR Algorithm (Ring of 5 Nodes) ---

[NODO 0 - UID 12] Inizia. Invia 'UID:12' a Nodo 1
[NODO 1 - UID 5] Inizia. Invia 'UID:5' a Nodo 2
[NODO 2 - UID 23] Inizia. Invia 'UID:23' a Nodo 3
[NODO 3 - UID 8] Inizia. Invia 'UID:8' a Nodo 4
[NODO 4 - UID 15] Inizia. Invia 'UID:15' a Nodo 0
[NODO 1 - UID 5] Ricevuto UID 12 MAGGIORE. Inoltro e divento
   passivo.
[NODO 2 - UID 23] Ricevuto UID 5 MINORE. Scarto.
[NODO 3 - UID 8] Ricevuto UID 23 MAGGIORE. Inoltro e divento
   passivo.
[NODO 4 - UID 15] Ricevuto UID 8 MINORE. Scarto.
[NODO 0 - UID 12] Ricevuto UID 15 MAGGIORE. Inoltro e
   divento passivo.
[NODO 2 - UID 23] Ricevuto UID 12 MINORE. Scarto.
[NODO 4 - UID 15] Ricevuto UID 23 MAGGIORE. Inoltro e
   divento passivo.
[NODO 1 - UID 5] Ricevuto UID 15 MAGGIORE. Inoltro e divento
   passivo.
[NODO 0 - UID 12] Ricevuto UID 23 MAGGIORE. Inoltro e
   divento passivo.
[NODO 2 - UID 23] Ricevuto UID 15 MINORE. Scarto.
>>> [NODO 2 - UID 23] HO RICEVUTO IL MIO UID! DIVENTO
   LEADER! <<<
[NODO 3 - UID 8] Acclamazione ricevuta! Leader e' 23.
   Inoltro e TERMINO.
[NODO 4 - UID 15] Acclamazione ricevuta! Leader e' 23.
   Inoltro e TERMINO.
[NODO 0 - UID 12] Acclamazione ricevuta! Leader e' 23.
   Inoltro e TERMINO.
[NODO 1 - UID 5] Acclamazione ricevuta! Leader e' 23.
   Inoltro e TERMINO.
[NODO 2 - UID 23] L'annuncio LEADER ha fatto il giro.
   TERMINO.
```

La chiusura a scarto è immacolata. L'UID asintoticamente più elevato era noto fin dall'inizio all'occhio clinico umano, ed una singola passata circolare è bastata allo script per terminare trionfante spezzando il Ring e ripulendo i file allocati in memoria virtuale temporanea.

5 Conclusioni e Sviluppi Futuri

Gli algoritmi studiati si applicano egregiamente all'emulazione IPC di Linux. Seppur originati nella teoria delle Reti Geografiche Distribuite a Commutazione di Pacchetto, si applicano identicamente a microstrutture informatiche parallele interne ad una CPU su ambienti Cloud.